

Esercizi di Autovalutazione — Linux e Bash

Autore: Prof. Danilo Croce — Corso di **Sistemi Operativi e Reti (SOR)**

Questo documento è un set di esercizi pratici a supporto della lezione “**Introduction-to-Linux-and-BASH**”.

Istruzioni: esegui tutto da terminale **Bash**. **!** Non usare ChatGPT o strumenti simili!

////////////////////////////////////

◆ 1) Navigazione e gestione file

1. Crea una cartella `esercizi_bash` e spostati al suo interno.
 2. All'interno crea due file di testo `testo1.txt` e `testo2.txt`.
 3. Apri `testo1.txt` con **vim** e scrivi **almeno due righe** di testo a tua scelta.
 4. Copia `testo1.txt` in `testo2.txt`, poi **inserisci una riga al centro** di `testo2.txt` con **vim**.
 5. Usa `diff` per mostrare le differenze tra i due file.
 6. Crea una sottocartella `backup/` e copia dentro **solo i file** `.txt`.
 7. Elenca tutti i `.txt` (anche nelle sottocartelle) con `find`.
 8. Crea un link simbolico a `testo1.txt` chiamato `link_testo`.
- ////////////////////////////////////

◆ 2) Visualizzazione e filtraggio del testo

Usa questo file di esempio `animali.txt` (campi separati da tab):

```
cane      mammifero  domestico
gatto     mammifero  domestico
leone     mammifero  selvatico
aquila    uccello  selvatico
pappagallo uccello  domestico
lucertola rettile  selvatico
```

1. Visualizza **le prime 3 righe** del file.
2. Visualizza **le righe 4–6** (usa insieme `head` e `tail`).
3. Trova tutte le righe che **non contengono** la parola `domestico`.
4. Estrai solo la **3^a colonna** (tab-separated) con `awk` e **conta** quante righe contengono `selvatico`.
5. Conta quante **classi** diverse compaiono nella **2^a colonna** (usa `awk`, `sort`, `uniq`, `wc`).
6. Con una pipeline `grep | sort | uniq | wc`, conta **quante righe** sono **esattamente uguali** a un input (esempio: `domestico`).

◆ 3) Manipolazione con `grep`, `awk`, `sed`

Usa questo file `studenti.tsv` (tab-separated):

```
id  nome    corso    voto
1   Marta   Programmazione  30
2   Paolo   Programmazione  28
3   Chiara  Sistemi  24
4   Luca    Sistemi  30
5   Anna    Programmazione  26
```

1. Estrai le righe con **3^a colonna = `Programmazione`**.
 2. **Conta** quante righe soddisfano la condizione.
 3. Con `sed`, sostituisci `Sistemi` con `Sistemi Operativi` **in tutto il file**.
 4. Mostra solo le righe con **`voto >= 28`** (`awk`).
 5. Salva l'output in `studenti_top.tsv`.
-

◆ 4) Contare, ordinare e combinare comandi

Usa questo file `parole.txt` (una parola per riga):

```
cane
gatto
gatto
leone
leone
leone
cane
orso
```

1. Ordina alfabeticamente e rimuovi i duplicati.
 2. **Conta** quante volte appare ciascuna parola.
 3. Trova la **parola più frequente**.
 4. Con `grep | sort | uniq | wc`, **conta** quante righe sono **uguali** a una parola fornita in input (esempio: `leone`).
 5. Selezionare tutte le righe che terminano con la sottostring `ne`.
 6. Selezionare tutte le righe che iniziano per `c` oppure `g`.
-

◆ 5) Variabili d'ambiente e PATH

1. Visualizza `$PATH` .
2. **Conta** quante **directory** sono nel `$PATH` .
3. Aggiungi **temporaneamente** una nuova directory a `$PATH` e verifica.
4. Crea una variabile `NOME` con il tuo nome e stampala con `echo` .
5. **Esporta** la variabile e verifica con `printenv` .

◆ 6) Redirezione di input/output e log

1. Esegui un comando che produca sia **stdout** sia **stderr** (es. `ls /etc /pippo`).
2. Redirigi **stdout** in `log_DATA.txt` e **stderr** in `log_ERR.txt` .
3. Verifica i contenuti dei due file.
4. Confrontali usando `less` o `diff` .
5. Unisci i due log in `log_TOT.txt` .

◆ 7) Processi e job control (singola shell)

Usa lo script visto a lezione: **salvalo come** `countdown.sh` **e rendilo eseguibile.**

```
#!/bin/bash

# Check if a parameter is given
if [ "$#" -ne 1 ]; then
    echo "Usage: $0 <starting_number>"
    exit 1
fi

START_NUM=$1

for i in $(seq $START_NUM -1 1); do
    echo $i
    sleep $((RANDOM % 3))
done
```

Esercizi:

1. Esegui **tre istanze** dello script con **tre numeri di partenza diversi** (es. 15, 10, 7).
2. **Metti in pausa** (Ctrl+Z) la **seconda** istanza.
3. **Termina** (kill) la **seconda** istanza in pausa.
4. Porta la **prima** istanza in **background** e la **terza** in **foreground** (usa `bg` / `fg`).
5. Verifica gli stati con `jobs` e `ps -ef | grep countdown.sh` .

Obiettivo: esercitarsi con **Ctrl+Z**, `jobs`, `bg`, `fg`, `kill` e con l'identificazione dei processi

tramite `ps`.

◆ 8) Processi e job control con `screen` (tre sessioni)

1. Crea **tre sessioni** `screen` distinte (ad es. `c1`, `c2`, `c3`).
2. In **ciascuna** sessione avvia `countdown.sh` con un numero di partenza diverso.
3. **Stacca** dalle sessioni (`detach`) e verifica con `screen -ls`.
4. **Riattacca** a `c2`, **metti in pausa** lo script e **termina** il job.
5. In `c1` lascia lo script in **background**, in `c3` in **foreground**.
6. **Chiudi** ordinatamente le tre sessioni.

Obiettivo: esercitarsi con **session persistence** e controllo dei job dentro `screen` (detach/reattach, gestione dei processi).

◆ 9) Esecuzione persistente con `nohup`

1. Esegui `countdown.sh` con `nohup` e **senza redirectione esplicita**.
2. Attendi la fine ed esamina il file generato `nohup.out`.
3. **Conta il numero di righe** presenti in `nohup.out`.
4. Ripeti l'esperimento con un **numero di partenza diverso** e osserva come `nohup.out` viene aggiornato.

Obiettivo: capire l'uso di `nohup` e la gestione del file di output di default.

◆ 10) Altri esercizi su processi

1. **Conta** il numero di processi associati **all'utente corrente**.
2. **Conta** quanti processi appartengono all'utente **root**.
3. Avvia `sleep 100 &`, verifica con `ps` e termina il processo.
4. Avvia un comando in foreground e poi **spostalo** in background usando `Ctrl+Z` + `bg`.

◆ 11) Ricerca file e contenuti

1. Trova in **tutto l'albero** a partire dalla directory corrente i file che si chiamano `*.log`.
2. Cerca nei file `.log` le righe che **non contengono** la parola `ERROR`.
3. Estrai dai `.log` tutte le righe che contengono `WARN` o `WARNING` e salva l'output in `warn.txt`.

◆ 12) Mini-lab combinato (grep | sort | uniq | wc)

Usa questo file `nomi.txt` (una parola per riga):

```
alfa
beta
gamma
beta
delta
gamma
gamma
alfa
```

1. Conta quante volte compare ciascun nome.
2. Ordina per **frequenza decrescente**.
3. Data una parola in input (es. `gamma`), conta **quante righe** sono **uguali** a quella parola usando **solo** una pipeline `grep | sort | uniq | wc` (nessun altro comando).

◆ 13) “Richiesta (quasi) stupida”: stampare un documento al contrario

Usa questo file `documento.txt` :

```
riga uno
riga due
riga tre
riga quattro
riga cinque
```

1. Stampa il file con **ordine delle righe invertito** (ultima → prima).
2. (Facoltativo) Stampa ogni riga con i **caratteri invertiti** (es. `uno` → `onu`).
3. (Facoltativo) Salva l'output invertito in `documento_reversed.txt` .

Suggerimento: esistono comandi standard per invertire **righe** e per invertire **caratteri**.

◆ 14) Log accessi — esercizi misti

Usa questo file `log_accessi.txt` :

```
2024-11-12 12:04:05 user1 LOGIN
2024-11-12 12:05:12 user2 LOGIN
2024-11-12 12:05:45 user1 LOGOUT
2024-11-12 12:06:03 user3 LOGIN
2024-11-12 12:08:25 user1 LOGIN
2024-11-12 12:09:01 user2 LOGOUT
```

1. Mostra solo le righe con `LOGIN`.
2. Conta quante volte **ogni utente** ha fatto `LOGIN`.
3. Trova l'**ultimo** `LOGIN` di `user1`.
4. Crea un file con **solo gli utenti** che hanno fatto almeno un `LOGIN`.
5. Ordina il file per **nome utente**.

◆ 15) Permessi con `chmod` (simbolico e ottale)

1. Crea `demo.txt` e verifica i permessi con `ls -l`.
2. Imposta permessi **simbolici**: `u=rw, g=r, o=` e verifica.
3. Ripeti con notazione **ottale**: imposta `640` e verifica.
4. Crea la dir `privata/`. Rimuovi `x` al **proprietario** e prova ad entrarci: cosa succede? Ripristina `x`.
5. (Facoltativo) Rendi eseguibile uno script `hello.sh` con `chmod +x` e avvialo con `./hello.sh`.

◆ 16) Tipo di file e ispezione binaria: `file`, `od`

Crea un piccolo binario di prova:

```
printf '\x41\x42\x0a' > bytes.bin
```

Visualizza `bytes.bin` con: `less bytes.bin`. Perché si riesce a leggere?

Qual'è il differenza tra

- `od -t x1 bytes.bin`
- `od -c bytes.bin`

◆ 17) Pager `less` (navigazione e ricerca)

1. Apri un file lungo con `less`.

2. Prova: **Space/f** (pagina avanti), **b** (indietro), **g** (inizio), **G** (fine).
3. Cerca con `/pattern` e ripeti con **n/N**. Cerca all'indietro con `?pattern`.
4. Mostra l'help con **h** e chiudi con **q**.
5. Apri `man bash` (che usa `less`) e cerca la sezione sulle **redirections**.

◆ 18) Stampare al contrario con `tac`

Usa `documento.txt`:

```
riga uno
riga due
riga tre
riga quattro
riga cinque
```

1. Stampa le **righe invertite** (ultima → prima) e salva in `reversed.txt`.
2. (Facoltativo) Combina con un altro comando per numerare le righe del risultato.

◆ 19) Selezione colonne con `cut` (TSV)

Usa `studenti.tsv` (tab-separated):

id	nome	corso	voto
1	Marta	Programmazione	30
2	Paolo	Programmazione	28
3	Chiara	Sistemi	24
4	Luca	Sistemi	30
5	Anna	Programmazione	26

1. Estrai **solo** le colonne `nome` e `voto` in `esiti.tsv`.
2. Estrai l'intera **3^a colonna** in `corsi.txt` e rimuovi i duplicati (usa anche `sort`, `uniq`).
3. Estrai le prime **N** righe di `esiti.tsv` senza usare un editor.

◆ 20) Trasformazioni semplici con `tr`

Crea `frasi.txt`:

```
CIAO MONDO
Linux  è  fantastico!!!
123abcDEF
```

1. Converti tutto in **minuscolo**.
2. **Rimuovi** tutte le cifre.
3. **Comprimi** (squeeze) gli spazi multipli in uno solo.
4. (Facoltativo) Mantieni solo lettere e spazi (filtra il resto).

◆ 21) Suddividere file con `split` e ricomporre

1. Genera `numeri.txt` con le righe 1..1000 (`seq`).
2. Spezza il file in blocchi da **100 righe** ciascuno.
3. Verifica quante parti sono state create e il numero di righe in ogni parte.
4. **Ricomponi** in `numeri_join.txt` e verifica con `wc -l` che corrisponda all'originale.

◆ 22) Creazione e gestione di directory e file annidati

1. Crea tre file di testo nella directory corrente:

```
touch file1.txt file2.txt file3.txt
```

2. Prova a copiare i tre file in una directory che non esiste ancora, ad esempio `backup/prova/` . Cosa succede?
3. Crea la directory annidata correttamente in un solo comando usando:

```
mkdir -p backup/prova
```

4. Ripeti la copia dei file e verifica il contenuto di `backup/prova` con `ls` .
5. Crea dentro `backup/prova` un sottodirectory `old/` e sposta lì solo `file1.txt` .
6. Torna nella directory principale e mostra ricorsivamente tutta la struttura con:

```
ls -R backup
```

7. (Facoltativo) Cancella l'intera directory `backup/` e tutto il suo contenuto con conferma esplicita.

◆ 23) Link simbolici con `ln -s`

1. Crea `foo/` e dentro `data.txt` con una riga di testo.
2. Crea in `.` un **link simbolico** `data_link` che punti a `foo/data.txt` .

3. Modifica l'originale e osserva cosa mostra il link.
 4. Elimina `foo/data.txt` : cosa mostra `cat data_link` ? Reinstalla il file e verifica di nuovo.
 5. (Facoltativo) Crea un link simbolico a **directory** e controlla il comportamento con `ls -l`.
-

◆ 24) Ricerca file con `find` (nome, tipo, azione)

1. Trova tutti i file con nome che **termina** in `.log` a partire dalla dir corrente.
 2. Trova tutte le **directory** chiamate `build`.
 3. Per ogni file `.log` trovato, **conta le righe** (usa `-exec ... {} +`).
 4. Trova i file **vuoti** e salvali in una lista `empty_files.txt`.
-

◆ 25) `ls` avanzato (filtri e ordinamenti)

Prepara una directory con file vari (diverse dimensioni e date).

1. Mostra **solo** i dettagli della directory corrente (non del contenuto).
 2. Visualizza elenco **ricorsivo** lungo (con permessi, dimensioni leggibili).
 3. Ordina per **dimensione** decrescente e mostra i **3** più grandi.
 4. Ordina per **data modifica** (più recenti in alto).
 5. Includi i file **nascosti** e spiega i campi della lunga lista (`-l`).
-

Bonus — Archiviazione con tar e gzip

1. Crea un archivio tar con **tutti i materiali** della cartella `esercizi_bash/`.
 2. Verifica il contenuto dell'archivio **senza estrarlo**.
-

 **Fine esercizi**